l'école d'ingénierie informatique

Étude sur la répartition des salaires

QUIMERC'H Pol & MERLE Antoine

Étude des salaires

Documentation

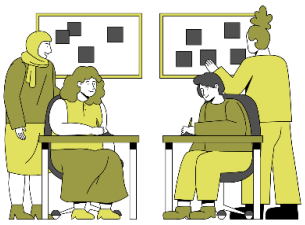
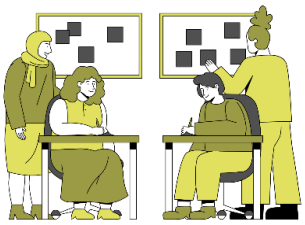


TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
CONTEXTE 3
OBJECTIF 3
VOCABULAIRE 3
INFORMATIONS 4
Analyse & Netoyage du dataset 5
Analyse des données à disposition 5
Préparation des données 5

Étude des salaires

Documentation



CONTEXTE

Lors de cette étude nous analysons un dataset d'offres d'emploi pour comprendre ce qui influence les salaires. Les données incluent des informations sur le type de poste (Data Scientist, Data Analyst, etc.), la note de l'entreprise, sa taille, son secteur d'activité, son âge (depuis la date de création).

OBJECTIF

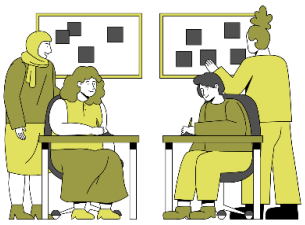
L'objectif est de créer un modèle capable de prédire le salaire moyen d'un poste en Data Science (par exemple) en fonction de plusieurs caractéristiques.

Nous testons quatre algorithmes de Machine Learning (Régression Linéaire, Arbres de Décision, Random Forest et Gradient Boosting) pour trouver celui qui prédit le mieux les salaires.

Nous visons un R^2 supérieur à 0.90. En plus de la prédiction, nous analysons les facteurs qui ont le plus d'impact sur les salaires (type de poste, secteur, taille de l'entreprise, etc.)

Étude des salaires

Documentation



INFORMATIONS

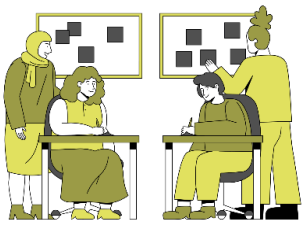
Le dataset utilisé dans cette étude contient 742 offres d'emploi, collectées sur différentes plateformes de recrutement.

Les informations principales sont :

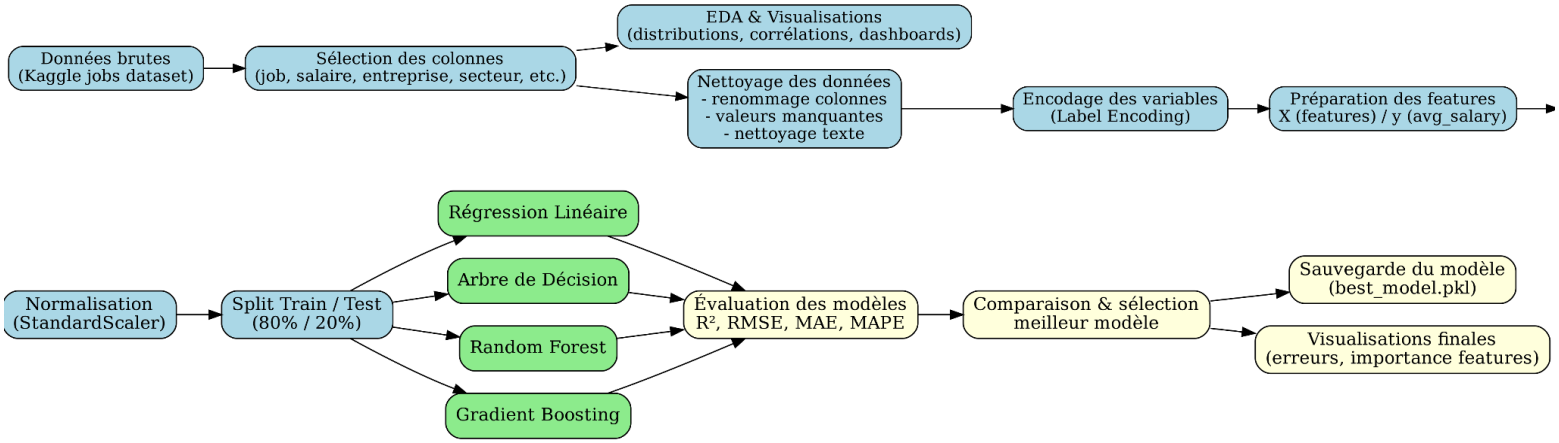
- le nom du poste (job_title)
- une catégorie simplifiée du poste (job_simp) qui regroupe les métiers similaires
- le salaire minimum, moyen et maximum proposés (min_salary, avg_salary, max_salary, en milliers de dollars)
- la note de l'entreprise sur 5 (rating)
- le nom et la taille de l'entreprise (company_name, size)
- le secteur d'activité (sector)
- le type de l'entreprise (type_of_ownership, entreprise privée, publique, etc.)
- l'âge de l'entreprise en années (age)

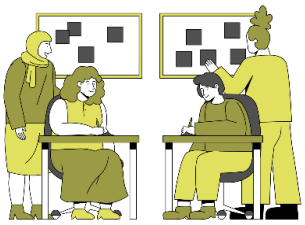
Étude des salaires

Documentation



Architecture





Analyse & Nettoyage du dataset

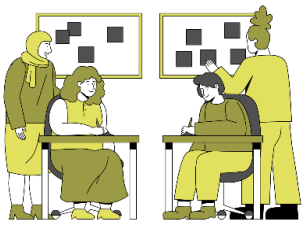
Data card :

Analyse des données à disposition

Le dataset initial est composé de 32 colonnes et 742 lignes. Nous avons décidé de sélectionner 12 colonnes afin d'optimiser les modèles et éviter un **overfitting** (surajustement).

Nous gardons les colonnes suivantes :

- jop_simp (type = object)
- rating (type = float64)
- company_name (type = object)
- size (type = object)
- founded (type = float64)
- industry (type = object)
- sector (type = object)
- min_salary (type = int64)
- avg_salary (type = float64)
- max_salary (type = int64)
- age (type = float64)
- type_of_ownership (type = object)



Préparation des données

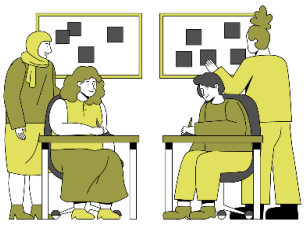
La première étape consiste à renommer les colonnes qui sont initialement composées d'espace et de majuscule. Une fois cette étape effectuée, nous identifions les colonnes catégorielles et numériques avec la méthode suivante :

```
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
```

Puis, nous transformons les colonnes catégorielles avec la fonction LabelEncoder()

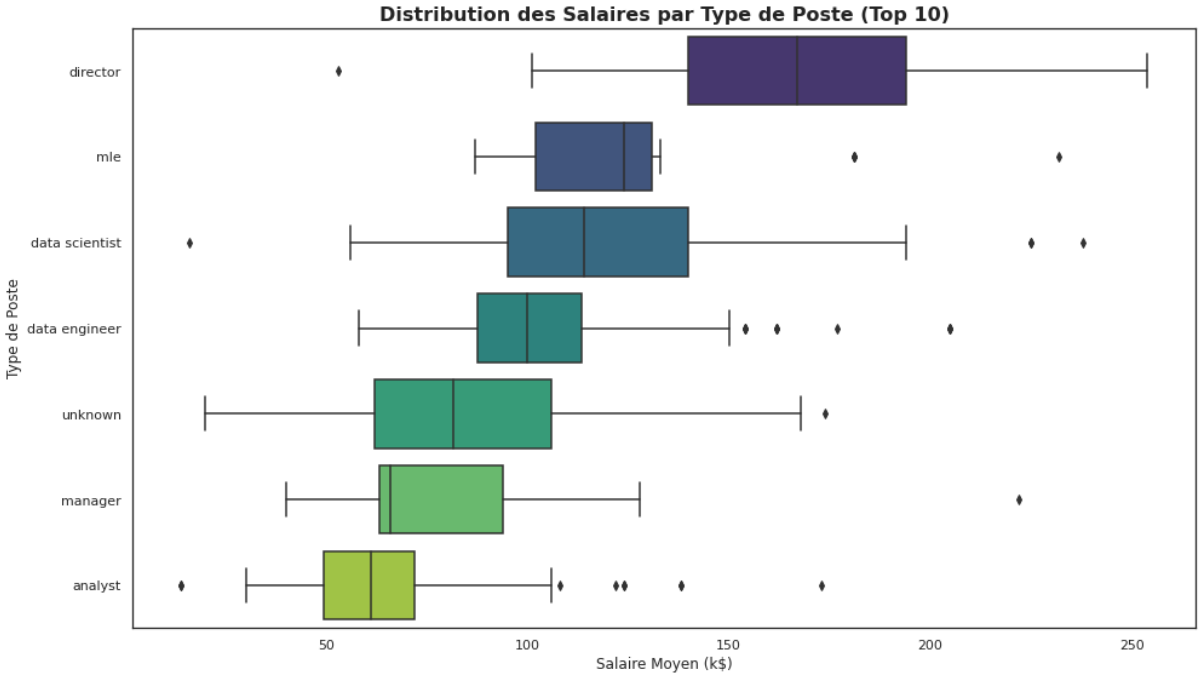
Ensuite, nous regardons la présence ou non de valeurs manquantes avec .isnull().sum(). Le dataset étant bien construit, aucune valeur manquante n'est déclarée.

```
✓ Colonnes renommées
✓ Données nettoyées
Colonnes catégorielles: ['job_simp', 'company_name', 'size', 'industry', 'sector', 'type_of_ownership']
Colonnes numériques: ['rating', 'founded', 'min_salary', 'avg_salary', 'max_salary', 'age']
Avant: 295 valeurs manquantes
Après: 0 valeurs manquantes
```



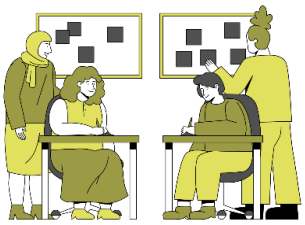
Analyse des données

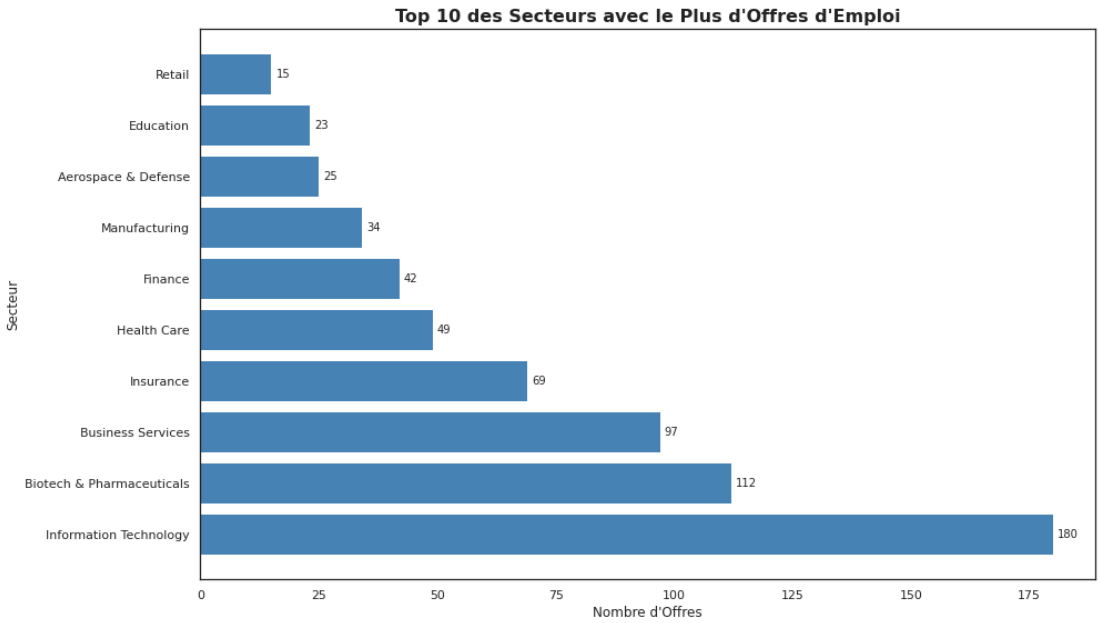
Maintenant que le dataset est prêt à être étudié, nous commençons par regarder la distribution des salaires par type de post



Étude des salaires

Documentation

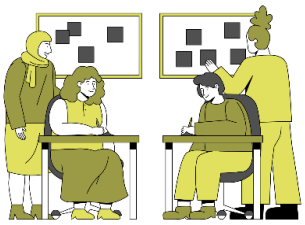


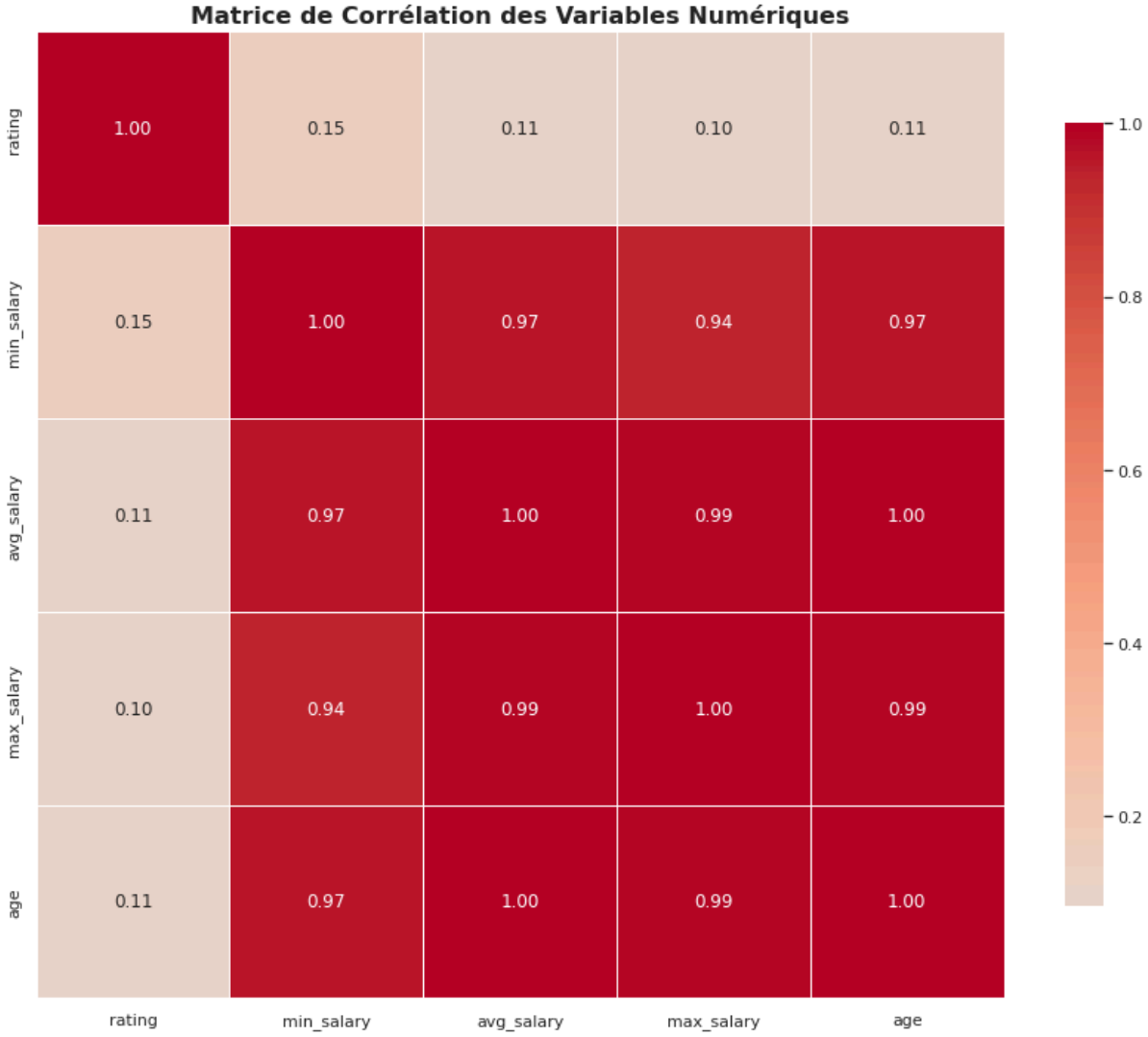


Ici, on observe la répartition des secteurs avec le plus d'offres d'emploi.
Le secteur avec le plus d'offres est le secteur technologique avec plus de 180 offres tandis que le commerce de détail (Retail) ne compte que 15 offres.

Étude des salaires

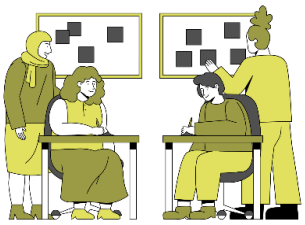
Documentation





Ensuite, nous avons généré la matrice de corrélation.
Il y a une très forte colinéarité entre les variables salariales, avec des coefficients entre 0.94 et 0.99. Cette corrélation s'explique par le fait que ces variables sont une dérivée du salaire.

En revanche, les notes des entreprises ont une faible corrélation avec les autres variables, ce qui suppose que la satisfaction / réputation d'une entreprise n'est pas directement liée à la rémunération proposée.



ENTRAÎNEMENT & MODÉLISATION

Model card :

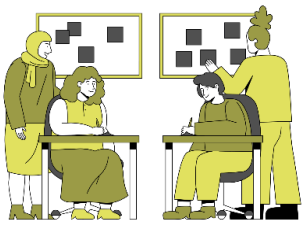
Préparation pour l'entraînement

Encodage des variables catégorielles

Tout d'abord nous commencons par la suppression de la colonne company_name qui contient trop de valeurs uniques et pourrait causer un suraprentissage. Ensuite, nous appliquons un **Label Encoder** sur chaque variable catégorielle restante '(`job_simp, sector, industry, size, type_of_ownership`).

Séparation des features et de la target

```
X = df[feature_cols].copy()
y = df['avg_salary'].copy()
X = X.drop('avg_salary', axis=1)

print(f"\nX shape: {X.shape}")
print(f"y shape: {y.shape}")
```

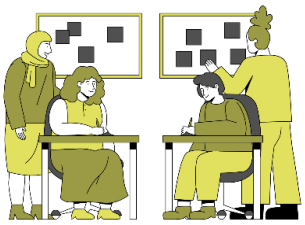


Standardisation & split train/test

Une fois les données séparées, nous avons normalisé x avec StandardScaler().

Après, nous séparons le dataset en un dataset d'entraînement et un dataset de test (taille du dataset de test : 20%) avec une sélection d'échantillons aléatoire (random_state=42).

```
print("CRÉATION DES SETS TRAIN/TEST")

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```



Entraînement des modèles & visualisations

Configuration des Modèles

Structure

Un **dictionnaire** nommé `models` contenant 4 algorithmes de régression avec leurs paramètres.

Les 4 Modèles

1. Linear Regression

- `LinearRegression()`
- Aucun paramètre spécifié

2. Decision Tree

- `DecisionTreeRegressor(random_state=42)`
- Paramètre : `random_state=42`

3. Random Forest

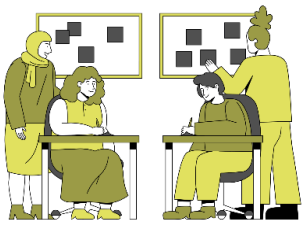
- `RandomForestRegressor(n_estimators=100, random_state=42)`
- Paramètres :
  - `n_estimators=100` (100 arbres)
  - `random_state=42`

4. Gradient Boosting

- `GradientBoostingRegressor(n_estimators=100, random_state=42)`
- Paramètres :
  - `n_estimators=100` (100 arbres)
  - `random_state=42`

Étude des salaires

Documentation

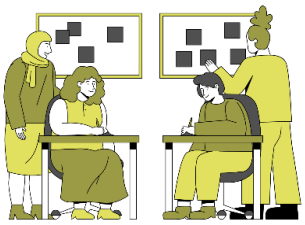


Observations

- ➔ Trois modèles utilisent `random_state=42`
- ➔ Deux modèles utilisent `n_estimators=100`
- ➔ Format dictionnaire avec noms en clés et objets modèles en valeurs
- ➔ Un modèle de régression linéaire et trois modèles basés sur les arbres

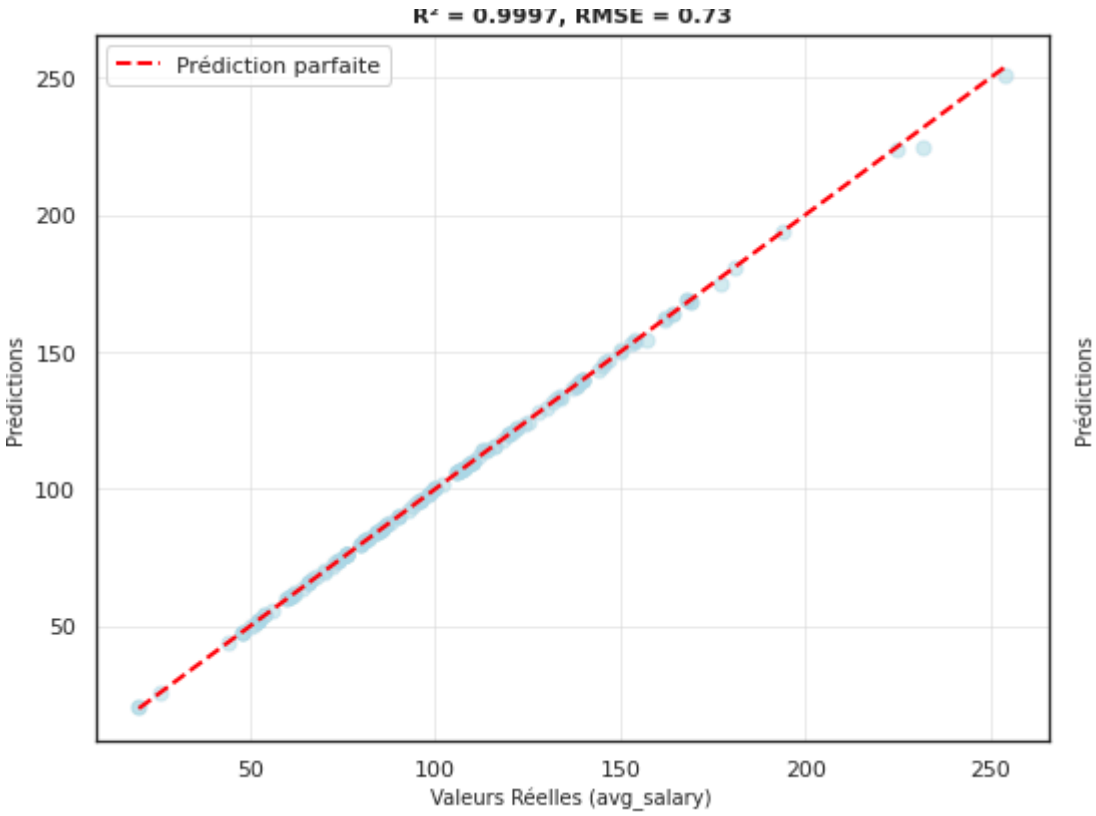
```
Données chargées:
X_train: (593, 10)
X_test: (149, 10)

Types de données dans X_train:
rating                       float64
founded                      float64
min_salary                   float64
max_salary                   float64
age                          float64
job_simp_encoded             float64
size_encoded                 float64
industry_encoded             float64
sector_encoded               float64
type_of_ownership_encoded    float64
dtype: object
```

```python
# Modèles
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100, random_state=42)
}
```



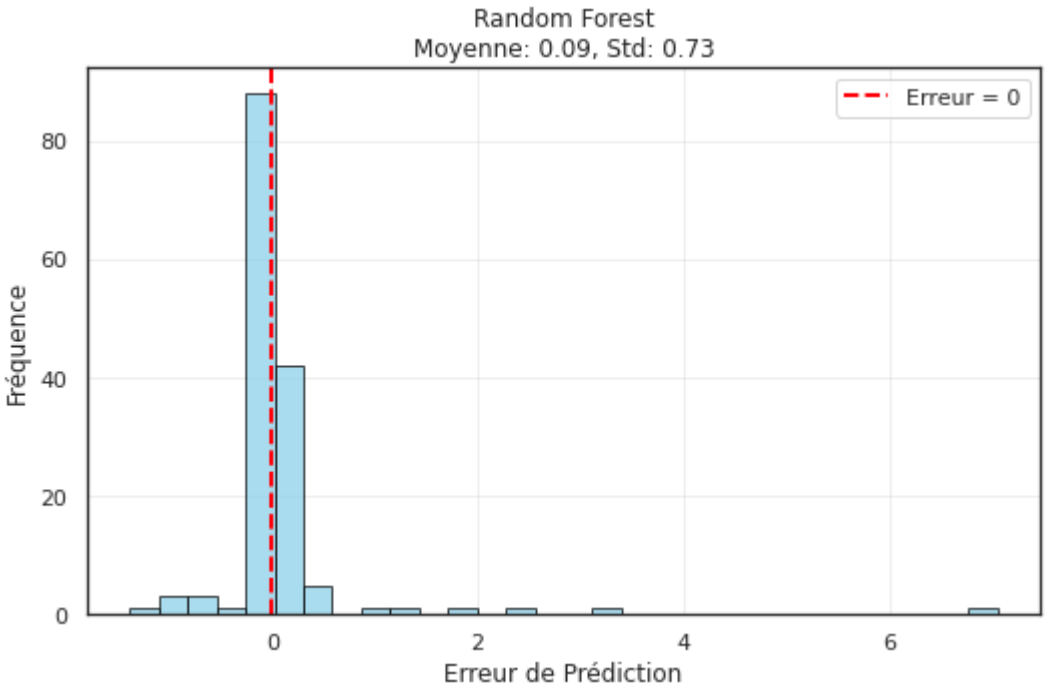
Graphes obtenus

Les graphiques suivants permettent d'analyser les performances des modèles après évaluation sur le jeu de test.



Ce graphique montre une excellente adéquation entre les valeurs réelles et les prédictions.

Les points sont quasiment tous alignés sur la droite de prédiction parfaite, ce qui indique que le modèle prédit les salaires avec une très grande précision.



Ce graphique montre que les erreurs de prédiction du modèle Random Forest sont fortement centrées autour de zéro, ce qui indique l'absence de biais significatif.

La moyenne des erreurs très proche de 0 (0,09) et l'écart-type faible (0,73) confirment que la majorité des prédictions sont très proches des valeurs réelles, avec seulement quelques erreurs plus élevées mais rares. Globalement, cela traduit un modèle stable, précis et fiable.